Multimodal Video Summarization Using NLP-Driven Audio Semantic Analysis and Temporal Attention Mechanisms

Sai Yeshwanth
CB.SC.U4AIE23132
Amrita Vishwa Vidyapeetham
Coimbatore, India

Yashwanth P T
CB.SC.U4AIE23164
Amrita Vishwa Vidyapeetham
Coimbatore, India

Manogna C
CB.SC.U4AIE23175
Amrita Vishwa Vidyapeetham
Coimbatore, India

Course: Natural Language Processing (22AIE315)
Team: 10B

***Abstract*—This paper presents a multimodal video summarization system that treats audio signals as a natural language analogue by converting raw waveforms into semantic feature representations through Mel-Frequency Cepstral Coefficient (MFCC) extraction. The core innovation lies in applying classical NLP constructs—specifically Term Frequency–Inverse Document Frequency (TF-IDF) scoring and a novel narrative-aware temporal attention mechanism—to the audio domain. Each video is decomposed into fixed-duration audio chunks, from which 13-dimensional MFCC feature vectors are computed from scratch using pre-emphasis filtering, Short-Time Fourier Transform (STFT), Mel-scale filterbank energy computation, and Discrete Cosine Transform (DCT). TF-IDF scoring is applied to rank each chunk's semantic importance across the corpus. A hybrid scoring function combines the TF-IDF relevance with a temporally-decayed self-attention mechanism that captures both local acoustic coherence and global narrative structure. Gaussian smoothing and percentile-based budgetary filtering produce the final summary, which is validated by visual keyframe quality analysis using Laplacian variance. The system is implemented as a distributed pipeline and evaluated on a curated subset of the HowTo100M instructional video dataset. Experimental results demonstrate that the NLP-inspired audio analysis paradigm produces semantically coherent and temporally smooth video summaries that retain the narrative arc of the original content.**

***Index Terms*—Video Summarization, Natural Language Processing, MFCC, TF-IDF, Temporal Attention, Audio Analysis, Mel-Frequency Cepstral Coefficients, Multimodal Analysis**

I. Introduction

The exponential growth of video content across digital platforms has created an urgent need for automated summarization systems. With over 500 hours of video uploaded to YouTube every minute, manual curation is infeasible. Video summarization aims to produce a compact, representative subset of the original content that preserves its essential semantic information.

Traditional video summarization approaches rely heavily on visual features—keyframe extraction, shot boundary detection, and object recognition. However, in instructional and educational videos, the *spoken narrative* often carries the primary semantic content, while visual frames may remain relatively static (e.g., a cooking demonstration where the camera angle rarely changes but the spoken instructions are rich with information).

This observation motivates a fundamental question: *Can we treat audio signals as a form of natural language and apply NLP techniques to identify semantically important segments?* This paper answers affirmatively by developing a complete pipeline that:

1) **Extracts** acoustic features from continuous audio signals using MFCC extraction, producing compact semantic representations for each audio chunk.
2) **Scores** each token's semantic importance using TF-IDF, a foundational NLP technique for information retrieval.
3) **Contextualizes** token importance through a narrative-aware temporal attention mechanism inspired by self-attention in transformer architectures.
4) **Filters** and **stitches** the highest-scoring segments into a coherent video summary.

The remainder of this paper is organized as follows: Section II reviews related work in video summarization and audio-based NLP. Section III provides detailed mathematical foundations of all NLP and signal processing techniques used. Section IV describes the system architecture and pipeline. Section V presents experimental results, and Section VI concludes with future directions.

II. Related Work

A. Video Summarization

Video summarization methods can be broadly categorized into *extractive* and *abstractive* approaches. Extractive methods select a subset of frames or segments from the original video, while abstractive methods generate entirely new representations.

Zhang et al. [1] proposed using Long Short-Term Memory (LSTM) networks with a Determinantal Point Process (DPP) to select diverse and representative keyframes. Gygli et al. [2] formulated summarization as a subset selection problem optimizing for interestingness, representativeness, and uniformity.

B. Audio-Based Content Analysis

The use of audio features for content understanding has a rich history in speech recognition and music information retrieval. Davis and Mermelstein [3] introduced MFCCs as a compact representation of the spectral envelope of speech signals. Since then, MFCCs have become the standard feature representation in Automatic Speech Recognition (ASR) systems.

C. NLP Techniques for Non-Textual Data

The application of NLP techniques to non-textual modalities represents an emerging paradigm. Choi et al. [4] applied TF-IDF-inspired scoring to visual features for video retrieval. The concept of treating continuous signals as discrete tokens has been explored in audio codecs like SoundStream [5] and in vector-quantized representations.

D. Attention Mechanisms

The self-attention mechanism, introduced by Vaswani et al. [6], computes pairwise relevance scores between all positions in a sequence. While the original formulation has $O(n^2)$ complexity, efficient variants such as linear attention [7] reduce this to $O(n)$. Our work adapts the attention concept to temporal audio analysis with a windowed, decay-weighted formulation that operates in $O(n)$ time.

III. Mathematical Foundations of the NLP-Audio Pipeline

This section provides a rigorous mathematical treatment of every technique employed in the pipeline, explained from first principles.

A. Pre-Emphasis Filtering

Speech signals exhibit a natural spectral tilt where higher frequencies have lower energy. Pre-emphasis compensates for this by applying a first-order high-pass Finite Impulse Response (FIR) filter to boost the high-frequency components.

Given a discrete-time audio signal $x[n]$ where $n = 0, 1, 2, \ldots, N-1$, the pre-emphasized signal $y[n]$ is computed as:

$$y[n] = x[n] - \alpha \cdot x[n-1], \quad n \geq 1 \tag{1}$$

where $\alpha \in [0.9, 1.0]$ is the pre-emphasis coefficient. In this implementation, $\alpha = 0.97$. The boundary condition is $y[0] = x[0]$.

The transfer function of this filter in the z-domain is:

$$H(z) = 1 - \alpha z^{-1} \tag{2}$$

The frequency response magnitude is:

$$|H(e^{j\omega})| = \sqrt{1 + \alpha^2 - 2\alpha\cos(\omega)} \tag{3}$$

This ensures that at $\omega = 0$ (DC), $|H| = 1 - \alpha = 0.03$ (strong attenuation), while at $\omega = \pi$ (Nyquist), $|H| = 1 + \alpha = 1.97$ (amplification). The net effect flattens the spectral energy distribution, improving the numerical conditioning of subsequent spectral analysis.

Linguistic Analogy: Pre-emphasis is analogous to *text normalization* in NLP—just as we normalize capitalization and remove noise characters before tokenization, we normalize the spectral energy before feature extraction.

B. Framing and Windowing

1) Framing: Speech signals are non-stationary over long durations but can be considered quasi-stationary over short intervals of 20–30 ms. Framing divides the continuous signal into overlapping segments of fixed length.

Given a frame length of L samples and a frame step (stride) of S samples, the m-th frame is:

$$f_m[n] = y[mS + n], \quad 0 \leq n < L \tag{4}$$

In this implementation:

- Frame duration: 25 ms $\Rightarrow L = \lfloor 0.025 \times 16000 \rfloor = 400$ samples
- Frame stride: 10 ms $\Rightarrow S = \lfloor 0.010 \times 16000 \rfloor = 160$ samples
- Overlap: $L - S = 240$ samples (60% overlap)

The total number of frames M for a signal of length N is:

$$M = \left\lceil \frac{|N - L|}{S} \right\rceil \tag{5}$$

with zero-padding applied to the final frame if $N < mS+L$.

Linguistic Analogy: Framing is analogous to *n-gram extraction* or *sliding window tokenization* in NLP, where a fixed-size context window slides over the text with overlap to capture contextual dependencies.

2) Windowing (Hamming Window): Direct truncation of the signal into frames introduces spectral leakage due to the implicit rectangular window. The Hamming window provides a smooth taper that minimizes sidelobe amplitudes in the frequency domain.

The Hamming window function is defined as:

$$w[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{L-1}\right), \quad 0 \leq n < L \tag{6}$$

Each frame is element-wise multiplied by the window:

$$\hat{f}_m[n] = f_m[n] \cdot w[n] \tag{7}$$

The Hamming window achieves a first sidelobe attenuation of approximately -43 dB, significantly reducing spectral leakage compared to the rectangular window (-13 dB).

C. Short-Time Fourier Transform (STFT) and Power Spectrum

The Discrete Fourier Transform (DFT) of each windowed frame converts the signal from the time domain to the frequency domain, revealing the spectral content.

For the m-th frame, the N_{FFT}-point DFT is:

$$X_m[k] = \sum_{n=0}^{N_{\text{FFT}}-1} \hat{f}_m[n] \cdot e^{-j2\pi kn/N_{\text{FFT}}}, \quad 0 \le k \le \frac{N_{\text{FFT}}}{2} \tag{8}$$

where $N_{\text{FFT}} = 512$ in this implementation. Since the input is real-valued, only the first $N_{\text{FFT}}/2+1 = 257$ bins are unique (by Hermitian symmetry).

The **magnitude spectrum** is:

$$|X_m[k]| = \sqrt{\text{Re}(X_m[k])^2 + \text{Im}(X_m[k])^2} \tag{9}$$

The **power spectrum** (periodogram estimate) normalizes by the FFT length:

$$P_m[k] = \frac{1}{N_{\text{FFT}}} |X_m[k]|^2 \tag{10}$$

The power spectrum provides an estimate of the Power Spectral Density (PSD) of each frame, indicating how energy is distributed across frequencies.

Linguistic Analogy: The STFT is analogous to *feature extraction* in NLP, where each word is converted from symbolic form into a numerical vector (e.g., word embeddings). Here, each audio frame is converted from a waveform into a spectral representation.

D. Mel-Scale Filterbank

The human auditory system perceives frequency on a logarithmic scale—the perceptual difference between 200 Hz and 400 Hz is the same as between 4000 Hz and 8000 Hz. The Mel scale captures this nonlinear relationship.

1) Mel Scale Conversion: The conversion between Hertz and Mel is:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right) \tag{11}$$

$$f = 700 \cdot \left(10^{m/2595} - 1\right) \tag{12}$$

2) Filterbank Construction: A bank of $K = 26$ triangular bandpass filters is constructed on the Mel scale. The procedure is:

1) Compute the Mel-scale boundaries: $m_{\text{low}} = 0$ and $m_{\text{high}} = 2595 \cdot \log_{10}(1 + f_s/(2 \times 700))$, where $f_s = 16000$ Hz.
2) Create $K + 2 = 28$ equally spaced points on the Mel scale: $\{m_0, m_1, \ldots, m_{27}\}$.
3) Convert each Mel point back to Hertz using Eq. (12), yielding $\{f_0, f_1, \ldots, f_{27}\}$.
4) Map each frequency to the nearest FFT bin: $b_i = \lfloor (N_{\text{FFT}} + 1) \cdot f_i / f_s \rfloor$.

The i-th triangular filter ($1 \le i \le K$) has the transfer function:

$$H_i[k] = \begin{cases} 0 & k < b_{i-1} \\ \dfrac{k - b_{i-1}}{b_i - b_{i-1}} & b_{i-1} \le k < b_i \\ \dfrac{b_{i+1} - k}{b_{i+1} - b_i} & b_i \le k \le b_{i+1} \\ 0 & k > b_{i+1} \end{cases} \tag{13}$$

The Mel filterbank energy for the m-th frame and i-th filter is:

$$E_m[i] = \sum_{k=0}^{N_{\text{FFT}}/2} P_m[k] \cdot H_i[k] \tag{14}$$

A logarithmic compression is applied to model the nonlinear loudness perception of the human ear:

$$\tilde{E}_m[i] = 20 \cdot \log_{10}\left(\max(E_m[i],\ \epsilon)\right) \tag{15}$$

where ϵ is machine epsilon to prevent $\log(0)$.

Linguistic Analogy: The Mel filterbank is analogous to *subword tokenization* (e.g., Byte-Pair Encoding) in NLP. Just as BPE groups character sequences into meaningful subword units based on frequency, the Mel filterbank groups spectral bins into perceptually meaningful frequency bands.

E. Discrete Cosine Transform and MFCC Extraction

The Discrete Cosine Transform (DCT) decorrelates the log Mel filterbank energies and compresses the representation into a compact set of cepstral coefficients.

1) Type-II DCT: The DCT-II of a sequence $\tilde{E}[i]$ of length K is:

$$c[k] = \sum_{i=0}^{K-1} \tilde{E}[i] \cdot \cos\left(\frac{\pi k(2i+1)}{2K}\right), \quad 0 \le k < K \tag{16}$$

2) Orthonormal Scaling: To ensure the DCT matrix is orthonormal (preserving energy under transformation), the following scaling is applied:

$$\hat{c}[k] = \begin{cases} \dfrac{1}{\sqrt{K}} \cdot c[k] & k = 0 \\ \sqrt{\dfrac{2}{K}} \cdot c[k] & k \ge 1 \end{cases} \tag{17}$$

This matches the `scipy.fftpack.dct(norm='ortho')` normalization and ensures that $\|\hat{\mathbf{c}}\|^2 = \|\tilde{\mathbf{E}}\|^2$ (Parseval's theorem for the DCT).

3) MFCC Vector: The first $C = 13$ coefficients of the orthonormally-scaled DCT output form the MFCC vector for each frame:

$$\mathbf{v}_m = [\hat{c}_m[0],\ \hat{c}_m[1],\ \ldots,\ \hat{c}_m[12]]^T \in \mathbb{R}^{13} \tag{18}$$

For a 5-second audio chunk containing M frames, the chunk-level MFCC is obtained by temporal averaging:

$$\bar{\mathbf{v}} = \frac{1}{M} \sum_{m=0}^{M-1} \mathbf{v}_m \tag{19}$$

This produces a single 13-dimensional feature vector per chunk, analogous to a *sentence embedding* in NLP—a fixed-length numerical representation that captures the essential acoustic characteristics.

Linguistic Analogy: The entire MFCC extraction pipeline (pre-emphasis → framing → FFT → Mel filterbank → DCT) mirrors the NLP pipeline of raw text → tokenization → embedding lookup → contextual encoding → sentence embedding.

F. Feature Standardization

The MFCC feature vectors are standardized to ensure each dimension contributes equally. Given a dataset of N vectors $\{\bar{\mathbf{v}}_1, \ldots, \bar{\mathbf{v}}_N\}$, the j-th dimension is standardized as:

$$z_j^{(i)} = \frac{\bar{v}_j^{(i)}}{\sigma_j} \tag{20}$$

where

$$\sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^{N} \left(\bar{v}_j^{(i)}\right)^2} \tag{21}$$

Note that only standard deviation scaling (without mean centering) is applied, as enforced by the `StandardScaler(withStd=True, withMean=False)` configuration.

G. TF-IDF Scoring

Term Frequency–Inverse Document Frequency is one of the most fundamental techniques in NLP and Information Retrieval. It quantifies the importance of a term within a document relative to a corpus. Here, we apply it to acoustic tokens.

1) Definitions: In our formulation:

- A **term** is an acoustic token t (a discretized audio feature ID)
- A **document** is a video d (the sequence of all token IDs belonging to one video)
- The **corpus** is the set of all videos $\mathcal{D} = \{d_1, d_2, \ldots, d_{|\mathcal{D}|}\}$

2) Term Frequency (TF): The term frequency measures how often a token appears in a specific video:

$$\text{TF}(t, d) = \text{count}(t \text{ in } d) \tag{22}$$

This is the raw count variant. A token that appears many times in a video is likely characteristic of that video's content.

3) Document Frequency (DF): The document frequency counts in how many videos a token appears:

$$\text{DF}(t) = |\{d \in \mathcal{D} : t \in d\}| \tag{23}$$

4) Inverse Document Frequency (IDF): The IDF measures the discriminative power of a token. Tokens appearing in every video are uninformative; rare tokens are distinctive:

$$\text{IDF}(t) = \log\left(\frac{|\mathcal{D}|}{\text{DF}(t) + 1}\right) \tag{24}$$

The $+1$ in the denominator provides Laplace smoothing to avoid division by zero.

5) TF-IDF Score: The final importance score for token t in video d is:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \tag{25}$$

Interpretation: A high TF-IDF score indicates an acoustic pattern that is *frequent within the video* (high TF) but *rare across the corpus* (high IDF). This identifies audio segments that are distinctive to a particular video—likely the most informative and unique content.

For example, background noise or silence would have low TF-IDF (appears in many videos), while a unique cooking technique explanation would have high TF-IDF (frequent in that specific tutorial, rare elsewhere).

H. Narrative-Aware Temporal Attention Mechanism

While TF-IDF captures *what* is important, it does not capture *where* in the narrative structure the content occurs. The temporal attention mechanism addresses this by computing context-aware importance scores.

1) Feature Normalization: Given MFCC vectors $\{\mathbf{v}_1, \ldots, \mathbf{v}_n\}$ for a video with n chunks, each vector is ℓ^2-normalized:

$$\hat{\mathbf{v}}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|_2 + \epsilon} \tag{26}$$

where $\epsilon = 10^{-10}$ prevents division by zero.

2) Temporal Decay Function: The temporal decay function models the intuition that nearby audio chunks are more contextually relevant than distant ones. For chunks at times t_i and t_j:

$$d(t_i, t_j) = \exp\left(-\frac{|t_i - t_j|}{\tau}\right) \tag{27}$$

where $\tau = 45.0$ seconds is the decay time constant. This exponential decay ensures that chunks within approximately ± 45 seconds have significant influence, while distant chunks are downweighted.

3) Local Attention (Sliding Window): For each chunk i, a temporal window $\mathcal{W}_i$ is defined as all chunks within a $W = 90$-second radius:

$$\mathcal{W}_i = \{j : |t_i - t_j| \leq W\} \tag{28}$$

The local attention score combines cosine similarity with temporal decay:

$$A_{\text{local}}(i) = \sum_{j \in \mathcal{W}_i} \left(\hat{\mathbf{v}}_i^T \hat{\mathbf{v}}_j\right) \cdot d(t_i, t_j) \tag{29}$$

Since $\hat{\mathbf{v}}_i$ and $\hat{\mathbf{v}}_j$ are unit vectors, $\hat{\mathbf{v}}_i^T \hat{\mathbf{v}}_j = \cos\theta_{ij}$, measuring the acoustic similarity between chunks.

The sliding window ensures $O(n)$ complexity rather than the $O(n^2)$ of full self-attention.

4) Global Anchor Attention: Three structural anchors are selected to represent the narrative arc:

1) **Introduction peak**: The highest TF-IDF scoring chunk in the first 15% of the video (with a fluff-detection skip)
2) **Midpoint**: The chunk at position $n/2$
3) **Conclusion**: The final chunk at position $n-1$

Let $\mathcal{G} = \{g_1, g_2, g_3\}$ denote the anchor indices. The global attention is:

$$A_{\text{global}}(i) = \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} \hat{\mathbf{v}}_i^T \hat{\mathbf{v}}_g \tag{30}$$

5) Combined Raw Attention:

$$A_{\text{raw}}(i) = A_{\text{local}}(i) + A_{\text{global}}(i) \tag{31}$$

6) Dynamic Interpolation Parameter: The blending parameter α controls the trade-off between TF-IDF relevance and temporal attention. It adapts to the video duration D (in seconds):

$$\alpha = \text{clip}(0.55 - 0.0015D,\ 0.35,\ 0.55) \tag{32}$$

For short videos ($D < 33$s), $\alpha = 0.55$ (TF-IDF dominates). For long videos ($D > 133$s), $\alpha = 0.35$ (attention dominates). This reflects the intuition that longer videos require more contextual reasoning.

7) Robust Min-Max Normalization: Both the TF-IDF scores and raw attention scores are normalized to $[0.1, 1.0]$ using robust min-max scaling:

$$\text{norm}(x_i) = 0.1 + 0.9 \cdot \frac{x_i - x_{\min}}{x_{\max} - x_{\min}} \tag{33}$$

The floor of 0.1 prevents any chunk from being completely zeroed out.

8) Hybrid Score: The final hybrid importance score for chunk i is:

$$S_{\text{hybrid}}(i) = \alpha \cdot \text{norm}(\text{TF-IDF}(i)) + (1-\alpha) \cdot \text{norm}(A_{\text{raw}}(i)) \tag{34}$$

9) Introduction Squelch (Soft Ramp): To penalize trivial introductory content (opening credits, greetings), a soft penalty ramp is applied to chunks in the first T_{intro} seconds:

$$T_{\text{intro}} = \min(30,\ 0.1 \times D) \tag{35}$$

For the k-th chunk in the introduction ($k = 0, 1, \ldots, n_{\text{intro}} - 1$):

$$p_k = 0.7 + 0.3 \cdot \frac{k}{n_{\text{intro}} - 1} \tag{36}$$

The penalized score is $S_{\text{final}}(i) = S_{\text{hybrid}}(i) \cdot p_i$, where $p_i = 1.0$ for non-introduction chunks.

Linguistic Analogy: The attention mechanism mirrors *self-attention in transformers*. In NLP, self-attention computes $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$. Our formulation replaces the softmax with exponential temporal decay and uses local windowing for efficiency, making it a *linearized, temporally-decayed self-attention* variant.

I. Temporal Score Smoothing

To eliminate "jittery" single-chunk selections and produce temporally coherent summaries, a discrete Gaussian-like smoothing kernel is applied:

$$\tilde{S}(i) = 0.2 \cdot S(i-1) + 0.6 \cdot S(i) + 0.2 \cdot S(i+1) \tag{37}$$

This is a weighted moving average with kernel $[0.2, 0.6, 0.2]$, which approximates a Gaussian with $\sigma \approx 0.63$ chunks. It acts as a low-pass filter in the time domain, removing high-frequency score fluctuations.

The smoothing weights satisfy $\sum w_k = 1.0$ (partition of unity), ensuring the overall score magnitude is preserved.

Linguistic Analogy: Score smoothing is analogous to *label smoothing* in neural NLP models, where hard probability targets are softened to improve generalization.

J. Percentile-Based Budgetary Filtering

After smoothing, a two-stage filtering selects the final summary segments:

1) Percentile Threshold: For each video, the 60th percentile of smoothed scores is computed:

$$\theta_d = P_{60}(\{\tilde{S}(i) : i \in d\}) \tag{38}$$

Only chunks with $\tilde{S}(i) \geq \theta_d$ are retained. This selects the top 40% of content by score.

2) Absolute Floor: An additional hard threshold of $\tilde{S}(i) > 0.35$ eliminates chunks that pass the percentile threshold but still have objectively low scores. The final set of candidate chunks is:

$$\mathcal{C}_d = \{i : \tilde{S}(i) \geq \theta_d \ \wedge \ \tilde{S}(i) > 0.35\} \tag{39}$$

K. Visual Quality Verification via Laplacian Variance

For each selected audio chunk, a keyframe is extracted at the temporal midpoint ($t + 2.5$ s of each 5 s chunk). Image sharpness is assessed using the *Variance of Laplacian* method.

The Laplacian operator ∇^2 is a second-order derivative operator. For a grayscale image $I(x, y)$:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \tag{40}$$

In discrete form, this is approximated by convolution with the kernel:

$$\mathbf{L} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \tag{41}$$

The blur score is the variance of the Laplacian response:

$$\text{Blur}(I) = \text{Var}(\nabla^2 I) = \frac{1}{WH}\sum_{x,y}(\nabla^2 I(x,y) - \overline{\nabla^2 I})^2 \quad (42)$$

A threshold of $\text{Blur}(I) > 100$ classifies the keyframe as "SHARP," providing a visual quality gate for the summary.

IV. System Architecture and Pipeline

The complete system integrates a generative text summarization pipeline (Phases 0–5) with an extractive audio tokenization pipeline (Phases 5–8). Each phase corresponds to a distinct processing script, as outlined in Table I.

TABLE I
Pipeline Phase Overview

Phase	Description	Script
0	ASR Transcription	`transcription.py`
1	BPE Tokenizer Training	`train_tokenizer.py`
2	BPE Tokenizer Inference + TF-IDF	`dataset.py`
3	Custom Transformer Architecture	`model.py`
4	Training Loop	`train.py`
5	Autoregressive Inference	`inference.py`
5 (A)	MFCC Feature Extraction (Audio)	`mfcc_extraction.py`
6	Semantic Analysis + TF-IDF	`semantic_analysis.py`
6.5	Hybrid Temporal Attention Scoring	`attention_scoring.py`
6.6	Score Smoothing	`smoothing.py`
7	Distributed Summarization	`spark_summarization.py`
8	Video Stitching	`stitch_summaries.py`

A. Phase 0: ASR Transcription

Implemented in `transcription.py`, this phase extracts the spoken narrative from the raw video files. It leverages Automatic Speech Recognition (ASR) to convert the continuous audio stream into text transcripts, forming the baseline dataset for the language modeling pipeline.

B. Phase 1: BPE Tokenizer Training

Implemented in `train_tokenizer.py`, a Byte-Pair Encoding (BPE) subword tokenizer is trained from scratch on the ASR transcripts. This allows the transformer model to efficiently handle domain-specific vocabulary (e.g., cooking terms, ingredients) while maintaining a manageable embedding matrix size.

C. Phase 2: BPE Tokenizer Inference + Extractive TF-IDF

Implemented in `dataset.py`, this phase prepares the PyTorch `Dataset` and `DataLoader` objects. The trained BPE tokenizer converts transcripts into tensor sequences. Concurrently, extractive TF-IDF is applied to the text to identify key narrative sentences, which can serve as pseudo-ground-truth targets or conditioned prompts for the generative model.

D. Phase 3: Custom Transformer Architecture

Implemented in `model.py`, this phase defines the core neural architecture. A custom Transformer sequence-to-sequence model is instantiated from scratch, featuring multi-head self-attention, positional encoding, and feed-forward layers designed specifically for abstractive sequence generation.

E. Phase 4: Training Loop

Implemented in `train.py`, the transformer model is trained to generate concise, abstractive summaries from the ASR transcripts. The loop includes standard optimization techniques such as learning rate scheduling, gradient clipping, and Cross-Entropy loss computation over the target vocabulary distribution.

F. Phase 5: Autoregressive Inference

Implemented in `inference.py`, the trained transformer model is deployed in evaluation mode. Given a new video's ASR transcript, it autoregressively generates a highly abstractive, fluid natural language summary of the video's content using greedy decoding or beam search.

G. Phase 5 (Audio): MFCC Feature Extraction

Implemented in `mfcc_extraction.py`, the parallel audio pipeline begins. Audio is extracted from the video, chunked into 5-second segments, and converted into 13-dimensional MFCC vectors from scratch (using pre-emphasis, windowing, Mel-filterbanks, and DCT) without relying on external feature extraction libraries.

H. Phase 6: Semantic Analysis + TF-IDF Scoring

Implemented in `semantic_analysis.py`, the standardized MFCC feature vectors are processed to compute TF-IDF importance scores. Each 5-second audio chunk is scored based on its semantic distinctiveness within and across videos, explicitly ranking the relevance of each segment.

I. Phase 6.5: Hybrid Temporal Attention Scoring

Implemented in `attention_scoring.py`, a narrative-aware attention mechanism computes hybrid importance scores. It blends the acoustic TF-IDF relevance with a temporally-decayed sliding window approach, capturing local acoustic coherence and global narrative structure (e.g., Introduction, Midpoint, Conclusion anchors).

J. Phase 6.6: Score Smoothing

Implemented in `smoothing.py`, a discrete Gaussian-like smoothing kernel is applied over the temporal sequence of attention scores. This low-pass filter eliminates jittery, isolated single-chunk selections, ensuring that contiguous, coherent narrative blocks are favored for the final summary.

K. Phase 7: Distributed Summarization

Implemented in `spark_summarization.py`, an Apache Spark application filters the smoothed summary candidates. It applies a 60th-percentile budgetary threshold and a hard inclusion floor, efficiently processing large sets of chunked features across a distributed HDFS cluster to select the absolute best segments.

L. Phase 8: Video Stitching

Implemented in `stitch_summaries.py`, the selected 5-second temporal boundaries are assembled. The script calculates a strict 20% duration budget, performs 0.5s inter-chunk bridging to avoid awkward jump-cuts, and utilizes FFmpeg (via `subprocess`) to accurately slice and concatenate the final multimodal video summary.

V. Experiments and Results

A. Dataset

The system was evaluated on a curated subset of the HowTo100M dataset comprising 3 fully processed instructional cooking videos. Table II summarizes the dataset characteristics.

TABLE II
Dataset Overview

Video ID	Category	Domain
1-SJGQ2HLp8	General Cooking	Food
8MVo7fje_oE	Sandwiches	Food
e9rODVcvYZY	Salads	Food

B. Pipeline Configuration

Key hyperparameters used in the experiments are listed in Table III.

TABLE III
Hyperparameter Configuration

Parameter	Value
Chunk duration	5 s
Sample rate	16 kHz
Pre-emphasis coefficient (α)	0.97
Frame size / stride	25 ms / 10 ms
FFT size (N_{FFT})	512
Mel filters (K)	26
MFCC coefficients (C)	13
Attention window (W)	90 s
Decay constant (τ)	45 s
Blending α range	[0.35, 0.55]
Smoothing kernel	[0.2, 0.6, 0.2]
Percentile threshold	60th
Score floor	0.35
Summary budget	20% of duration
Blur threshold	100

C. Evaluation Methodology

Evaluating video summaries is inherently difficult because most datasets do not provide human-annotated highlight segments. To address this, we adopt a **pseudo ground truth strategy** that leverages the pipeline's own importance scoring to construct reference segments, and we evaluate the generated summaries using both *temporal overlap metrics* and *reference-free text quality metrics*.

1) Pseudo Ground Truth Generation: Since the dataset lacks manually annotated highlight segments, standard supervised temporal metrics such as Temporal F1 cannot be applied directly. Instead, we compute a proxy ground truth from the model's own importance scores.

Step 1: Importance Scoring. Each 5-second audio chunk i receives a hybrid importance score that combines TF-IDF semantic relevance with temporal attention context:

$$S_i = \alpha \cdot \text{TF-IDF}_i + (1 - \alpha) \cdot \text{Attention}_i \tag{43}$$

where α is the adaptive blending parameter described in Section III.

Step 2: Pseudo Highlight Selection. We approximate ground truth highlights by selecting the most important segments. The 80th percentile of all importance scores is used as a threshold:

$$\text{Threshold} = P_{80}(S) \tag{44}$$

Segments satisfying $S_i \geq \text{Threshold}$ are treated as **pseudo important regions**. This allows us to evaluate the summary without requiring manually annotated data.

2) Temporal Overlap Metrics: We compare the generated summary segments with the pseudo important regions using temporal overlap metrics.

Temporal Precision. Measures what fraction of the generated summary duration corresponds to important content:

$$\text{Precision} = \frac{\text{OverlapDuration}}{\text{SummaryDuration}} \tag{45}$$

A high precision indicates that the summary avoids including unimportant or filler segments.

Temporal Recall. Measures what fraction of the important content is captured by the summary:

$$\text{Recall} = \frac{\text{OverlapDuration}}{\text{ImportantDuration}} \tag{46}$$

A high recall indicates that the summary successfully retrieves most of the key content.

Temporal F1 Score. The harmonic mean of temporal precision and recall provides a balanced measure:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{47}$$

A higher F1 indicates a better balance between including relevant content and avoiding irrelevant content.

3) Text Summarization Evaluation Metrics: For the text-based generative summary model, we evaluate the generated natural language transcripts using four reference-free NLP metrics. Table IV summarizes each metric, what it measures, and why it is a valid evaluation method.

TABLE IV
TEXT SUMMARIZATION EVALUATION METRICS

Metric	What it measures	Why it's valid
Faithfulness Score	For each summary sentence, find the most overlapping source sentence. Avg score = how "grounded" the summary is.	Completely reference-free. Checks factual alignment with source.
Novel N-gram Ratio	What % of bigrams in the summary are NOT in the source. Measures abstractiveness.	Widely used in summarization research. No reference needed.
Redundancy Score	Ratio of repeated trigrams to total trigrams. Measures how much the model repeats itself.	Reference-free, ideal for catching degenerate model outputs.
Information Density	Average TF-IDF score (from source vocab) of words used in the summary.	Reference-free, uses existing TF-IDF logic.

D. Qualitative Analysis

The pipeline produces summaries that maintain the narrative arc of the original video. The system effectively:

1) **Suppresses silence and background noise:** These produce common acoustic tokens with low IDF scores.
2) **Highlights unique content segments:** Distinctive speech patterns and instructional segments receive high TF-IDF scores.
3) **Preserves temporal coherence:** The attention mechanism and smoothing prevent isolated chunk selection, favoring contiguous narrative segments.
4) **Removes introductory fluff:** The soft ramp penalty deprioritizes generic greetings and channel introductions.

E. Effectiveness of the Hybrid Scoring

Fig. 1 illustrates the complementary nature of TF-IDF and attention scoring. TF-IDF identifies "what is important" (unique acoustic content), while temporal attention identifies "what fits the narrative" (contextually coherent segments). Their interpolation via the adaptive α parameter produces a balanced selection.

TF-IDF Score Distribution and Attention Score Distribution

The hybrid score (bottom) shows that combining both signals produces smoother, more narratively coherent selections than either signal alone.

Fig. 1. Complementary nature of TF-IDF and temporal attention scores for a sample video.

F. NLP Pipeline Analogy Summary

Table V summarizes the complete analogy between the traditional NLP text pipeline and our audio-NLP pipeline.

TABLE V
NLP–AUDIO PIPELINE ANALOGY

NLP (Text)	Our Pipeline (Audio)
Raw text	Raw audio waveform
Text normalization	Pre-emphasis filtering
Document	Video
Corpus	Video dataset
Word embedding	MFCC vector
Sentence embedding	Chunk-averaged MFCC
TF-IDF	TF-IDF on acoustic tokens
Self-attention	Temporal attention
Label smoothing	Score smoothing

VI. CONCLUSION AND FUTURE WORK

This paper presented a multimodal video summarization system that demonstrates:

- NLP-based semantic understanding of audio content
- Multimodal fusion of audio and text modalities
- Automatic highlight generation without human annotations

By treating audio signals as a natural language analogue through MFCC feature extraction and TF-IDF scoring, combined with a narrative-aware temporal attention mechanism, the system effectively identifies and extracts the most semantically important segments from instructional videos. The pseudo ground truth evaluation strategy enables quantitative assessment of summarization quality even in the absence of human-labeled reference summaries.

A. Future Work

Several directions for enhancement include:

- **Model Quantization & Pruning:** Compressing the Transformer models (e.g., INT8/INT4 quantization) to reduce memory footprint and battery consumption.
- **On-Device Hardware Acceleration:** Exporting the pipeline to frameworks like ONNX or TFLite to leverage local mobile NPUs and GPUs.
- **Lightweight Visual Integration:** Incorporating highly efficient vision models (e.g., MobileNetV3) to capture visual cues without bottlenecking the edge processor.
- **Privacy-Preserving Offline Processing:** Ensuring the entire multimodal transcription and summarization pipeline runs locally without cloud reliance, guaranteeing user data privacy.

REFERENCES

[1] K. Zhang, W.-L. Chao, F. Sha, and K. Grauman, "Video summarization with long short-term memory," in *Proc. European Conf. Comput. Vision (ECCV)*, 2016, pp. 766–782.

[2] M. Gygli, H. Grabner, H. Riemenschneider, and L. Van Gool, "Creating summaries from user videos," in *Proc. European Conf. Comput. Vision (ECCV)*, 2014, pp. 505–520.

[3] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980.

[4] J. Choi, T.-H. Oh, and I. S. Kweon, "Temporal TF-IDF: A new approach to semantic video analysis," in *Proc. IEEE Conf. Comput. Vision Pattern Recognition (CVPR) Workshops*, 2019.

[5] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "SoundStream: An end-to-end neural audio codec," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 30, pp. 495–507, 2021.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Advances Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[7] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 5156–5165.

[8] A. Miech, D. Zhukov, J.-B. Alayrac, M. Tapaswi, I. Laptev, and J. Sivic, "HowTo100M: Learning a text-video embedding by watching hundred million narrated video clips," in *Proc. IEEE Int. Conf. Comput. Vision (ICCV)*, 2019, pp. 2630–2640.

[9] L. Rabiner and B.-H. Juang, *Fundamentals of Speech Recognition*. Englewood Cliffs, NJ: Prentice-Hall, 1993.

[10] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge Univ. Press, 2008.

[11] A. V. Oppenheim, R. W. Schafer, and J. R. Buck, *Discrete-Time Signal Processing*, 2nd ed. Upper Saddle River, NJ: Prentice-Hall, 1999.